pfi

Python for Fixed Income

COURSE 2 · BOND MATH

Case Study: A New Issue Calendar

WEEK 1 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem definition: eight new issues, five asks

The Desk Lead's Fifth Note

Eight new issues price today, all settling on 30 September 2026. Before the calendar goes round: what does each yield at its re-offer price? Price all eight at one yield, so coupon and term compare on equal terms. On the grid of eighths, which coupon would price each one nearest par at its own yield, and what is one eighth of coupon worth in price? Last, send it as a workbook whose yields and prices Excel works out itself. Describe the calendar: no view on any issue.

The Data

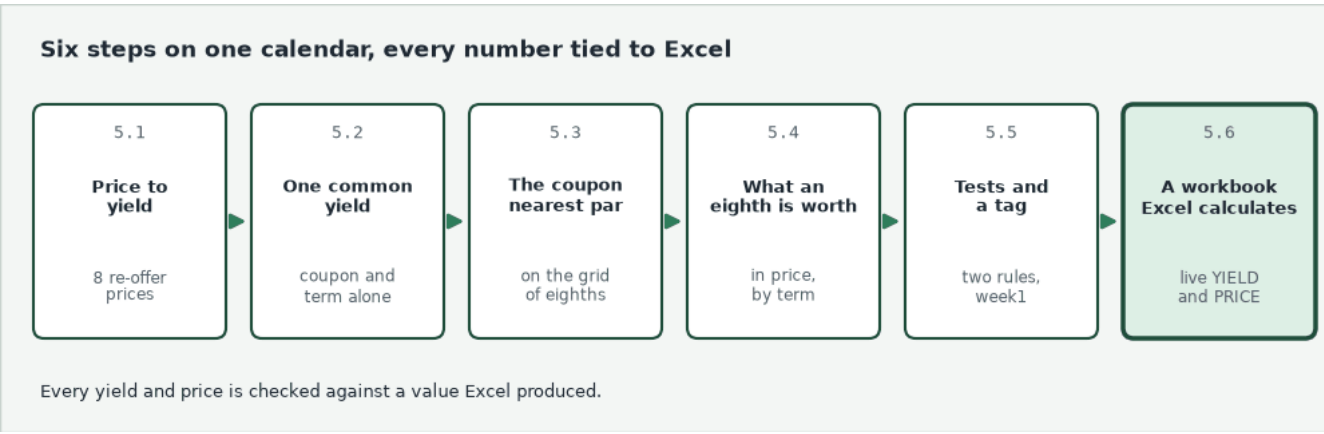
CUSIP	Issuer	Rating	Coupon (%)	Years	Maturity	Re-offer price
99010KZA6	Thalostrin Finance	BBB+	5.125	5	2031-09-30	99.874
99011LZA3	Galvostrin Resources	BBB+	5.625	10	2036-09-30	99.512
99012MZA0	Hulvimer Pipeline	BBB	5.5	7	2033-09-30	99.760
99013NZA7	Kezerone Paper	A-	4.625	3	2029-09-30	99.942
99014OZA4	Elvorin Medical	BB-	7.875	8	2034-09-30	100.000
99016QZA7	Corvolvey Systems	BB+	7	5	2031-09-30	99.000
99017RZA4	Rulvolvey Industrial	BBB-	6	30	2056-09-30	98.975
99020UZA2	Morvithe Logistics	B	8.75	6	2032-09-30	98.500

The re-offer price is the price the new bonds are sold at. Each bond settles on its issue date, so nothing has accrued and notebook 02's coupon-date pricing applies exactly.

Invented issuers and CUSIPs: fixed-rate bullets, coupons twice a year, 30/360.

You are the desk's analyst: answer the five asks with your module, and tie every number to Excel.

How we will work, and the calendar checked



Eight prices in; a workbook Excel calculates out.

```
..........   [100%]
11 passed in 0.50s
```

Six Predicts on the way: decide before you run the cell.

Checked Before It Is Used

```
assert calendar['cusip'].is_unique, 'two issues share a CUSIP'
assert ((calendar['coupon_pct'] * 8) % 1 == 0).all(), 'a coupon is off the grid of eighths'
print('Eight different CUSIPs, every coupon a whole number of eighths')
Eight different CUSIPs, every coupon a whole number of eighths
```

Job	Python	Excel	Result
Count the different CUSIPs, A1:A8	`calendar['cusip'].nunique()`	`=SUMPRODUCT(1/COUNTIF(A1:A8,A1:A8))`	8
Coupon on the grid	`(5.125 * 8) % 1`	`=MOD(5.125*8,1)`	0

5.1 Yields from re-offer prices

```
thalostrin = calendar.iloc[0]
thalostrin_yield_pct = bondmath.yield_from_price(thalostrin['coupon_pct'],
                                                 thalostrin['reoffer_price'], thalostrin['years'])
print(f"Thalostrin yields {thalostrin_yield_pct:.4f} percent at {thalostrin['reoffer_price']}")
Thalostrin yields 5.1539 percent at 99.874
```

A little over its 5.125 coupon: a price under 100 means a yield over the coupon, as notebook 03 found.

```
calendar['yield_pct'] = [bondmath.yield_from_price(c, p, t) for c, p, t in
                         zip(calendar['coupon_pct'], calendar['reoffer_price'], calendar['years'])]
calendar['gap_bp'] = (calendar['yield_pct'] - calendar['coupon_pct']) * 100
(calendar[['ticker', 'years', 'reoffer_price', 'yield_pct', 'gap_bp']].style
    .format({'reoffer_price': '{:.3f}', 'yield_pct': '{:.4f}', 'gap_bp': '{:.1f}'})
    .background_gradient(subset=['gap_bp'], cmap='Greens')
    .hide(axis='index'))
```

ticker	years	reoffer_price	yield_pct	gap_bp
THAN	5	99.874	5.1539	2.9
GALN	10	99.512	5.6897	6.5
HULR	7	99.760	5.5418	4.2
KEZE	3	99.942	4.6459	2.1
ELVN	8	100.000	7.8750	-0.0
CORY	5	99.000	7.2420	24.2
MORE	6	98.500	9.0797	33.0

From 4.6459 percent (Kezerone, 3 years) to 9.0797 (Morvithe, 6 years), and 2.1 to 33.0 bp over the coupon. Elvorin, at exactly 100, yields its coupon: its -0.0 is the search stopping a hair under 7.875.

```
tvm = pd.read_csv('bondmath_excel_reference_tvm.csv')
excel_yields = tvm[tvm['function'] == 'CALENDAR_YIELD'].head(8)
calendar['excel_yield_pct'] = (excel_yields['excel_value'] * 100).to_numpy()
largest_gap = (calendar['yield_pct'] - calendar['excel_yield_pct']).abs().max()
print(f'Largest gap to Excel: {largest_gap:.1e} percent')
Largest gap to Excel: 6.1e-09 percent
```

"YIELD" takes the coupon as a decimal and returns one: times 100 for percent.

Job	Excel	Result
Thalostrin's yield	`=YIELD(DATE(2026,9,30),DATE(2031,9,30),5.125/100,99.874,100,2,0)`	0.0515390782041645

Eight yields, each within 6.1e-09 percent of Excel. Now take credit out: one yield for all.

5.2 Prices at a common yield: only coupon and term left

Each yield mixes the issuer's credit with the bond's term. Price all eight at one round yield, 5.50 percent, a what-if and not a market level, and only coupon and term are left. A coupon above 5.50 then prices over 100, one below it under 100 (notebook 03).

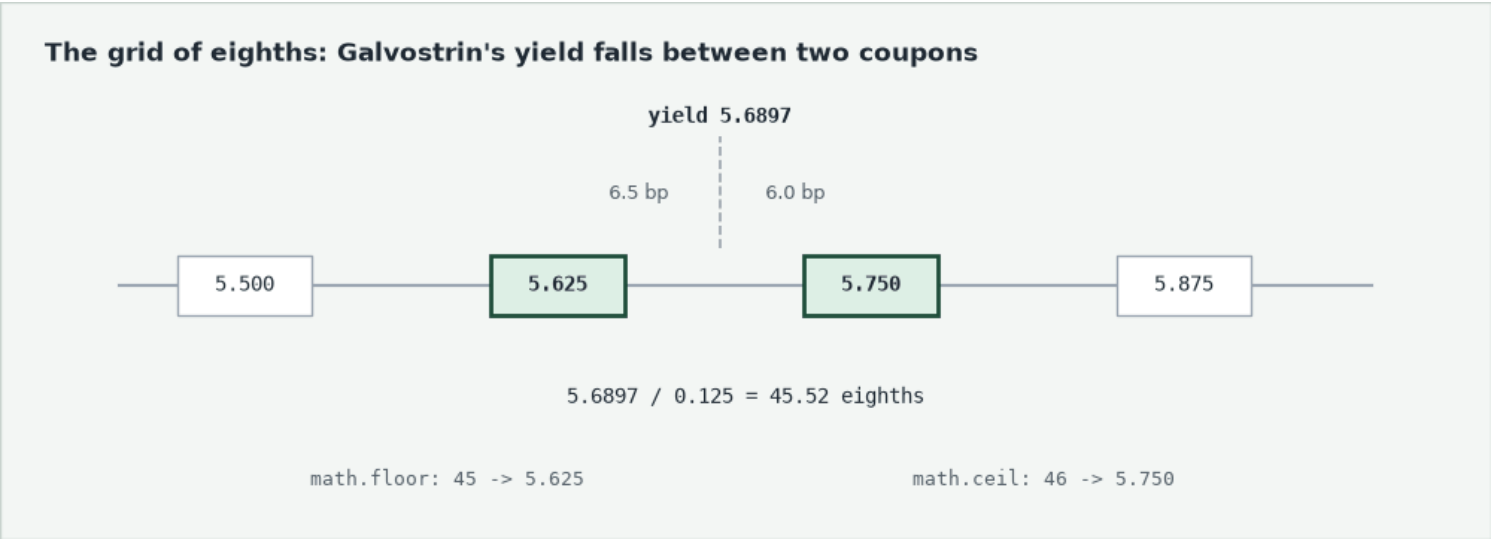
All Eight at 5.50 Percent, Against Excel's PRICE

```
calendar['price_at_5_50'] = [bondmath.price_from_yield(c, common_yield_pct, t)
                             for c, t in zip(calendar['coupon_pct'], calendar['years'])]
excel_prices = tvm[tvm['function'] == 'CALENDAR_PRICE'].head(8)
calendar['excel_price_at_5_50'] = excel_prices['excel_value'].to_numpy()
largest_gap = (calendar['price_at_5_50'] - calendar['excel_price_at_5_50']).abs().max()
print(f'Largest gap to Excel: {largest_gap:.1e} per 100 of face')
Largest gap to Excel: 1.1e-13 per 100 of face
```

Excel side by side

Job	Python	Excel
Rulvolvey at 5.50	`bondmath.price_from_yield(6.0, 5.5, 30)`	`=PRICE(DATE(2026,9,30),DATE(2056,9,30),6/100,5.5/100,100,2,0)`
Corvolvey at 5.50	`bondmath.price_from_yield(7.0, 5.5, 5)`	`=PRICE(DATE(2026,9,30),DATE(2031,9,30),7/100,5.5/100,100,2,0)`

5.3 The coupon nearest par, on a grid of eighths



Coupons are set in eighths: 5.500, 5.625, 5.750, and so on. A coupon equal to the yield prices at 100 (notebook 03), but a yield is rarely a whole number of eighths. So take the two grid coupons either side of the yield, and keep the one that prices nearer 100.

Rounding to the Grid

```
import math
galvostrin = calendar[calendar['ticker'] == 'GALN'].iloc[0]
eighths = galvostrin['yield_pct'] / 0.125
low_coupon, high_coupon = math.floor(eighths) * 0.125, math.ceil(eighths) * 0.125
print(f"{galvostrin['yield_pct']:.4f} percent is {eighths:.2f} eighths:",
      f'{low_coupon} or {high_coupon}')
5.6897 percent is 45.52 eighths: 5.625 or 5.75
```

The two-coupon test as a function

The Grid Coupon for Every Issue

```
def nearest_par_coupon(yield_pct, years):
    eighths = yield_pct / 0.125
    candidates = [math.floor(eighths) * 0.125, math.ceil(eighths) * 0.125]
    distances = [abs(bondmath.price_from_yield(c, yield_pct, years) - 100) for c in candidates]
    return candidates[distances.index(min(distances))]
```

```
calendar['par_coupon_pct'] = [nearest_par_coupon(y, t) for y, t in
                              zip(calendar['yield_pct'], calendar['years'])]
calendar['eighths_up'] = ((calendar['par_coupon_pct'] - calendar['coupon_pct']) / 0.125).round()
(calendar[['ticker', 'coupon_pct', 'yield_pct', 'par_coupon_pct', 'eighths_up']].style
    .format({'coupon_pct': '{:.3f}', 'yield_pct': '{:.4f}', 'par_coupon_pct': '{:.3f}',
             'eighths_up': '{:.0f}'})
    .background_gradient(subset=['eighths_up'], cmap='Greens')
    .hide(axis='index'))
```

Excel side by side: FLOOR and CEILING

Job	Python	Excel	Result
Round down to the grid	`math.floor(5.69 / 0.125) * 0.125`	`=FLOOR(5.69,0.125)`	5.625
Round up to the grid	`math.ceil(5.69 / 0.125) * 0.125`	`=CEILING(5.69,0.125)`	5.75

5.4 An eighth on every issue, and 5.5 two rules

An Eighth on Every Issue, Against Excel

```
calendar['step_value'] = [bondmath.price_from_yield(c + 0.125, common_yield_pct, t)
                          - bondmath.price_from_yield(c, common_yield_pct, t)
                          for c, t in zip(calendar['coupon_pct'], calendar['years'])]
excel_stepped = tvm[tvm['function'] == 'CALENDAR_PRICE'].tail(8)['excel_value'].to_numpy()
excel_steps = excel_stepped - calendar['excel_price_at_5_50']
print(f"Largest gap to Excel: {(calendar['step_value'] - excel_steps).abs().max():.1e}")
Largest gap to Excel: 2.8e-14
```

Excel side by side

```
An eighth on Rulvolvey
=PRICE(DATE(2026,9,30),DATE(2056,9,30),6.125/100,5.5/100,100,2,
0)-PRICE(DATE(2026,9,30),DATE(2056,9,30),6/100,5.5/100,100,2,0)
The extra coupons alone
=-PV(5.5/100/2,2*30,0.125/2,0)
```

Two facts carried the case: a coupon equal to the yield prices at 100 (5.3), and price falls as yield rises (notebook 03, behind every yield in 5.1). They hold for every bond, so they go in the test file.

```
%%writefile -a test_bondmath.py
def test_par_bond_prices_at_100():
    for frequency in (1, 2, 4):
        for years in (1, 2, 5, 10, 30):
            assert bondmath.price_from_yield(5.5, 5.5, years, frequency) == pytest.approx(100, abs=1e-9)
def test_price_falls_as_yield_rises():
    prices = [bondmath.price_from_yield(6.0, yield_pct, 10, 2) for yield_pct in (2, 4, 6, 8, 10, 12)]
    for earlier, later in zip(prices, prices[1:]):
        assert later < earlier
```

The first prices fifteen par bonds: five terms at three frequencies. In the second, "prices[1:]" is the list without its first price, so "zip" pairs each price with the next.

5.5 Tests pass, a commit, and the week tagged

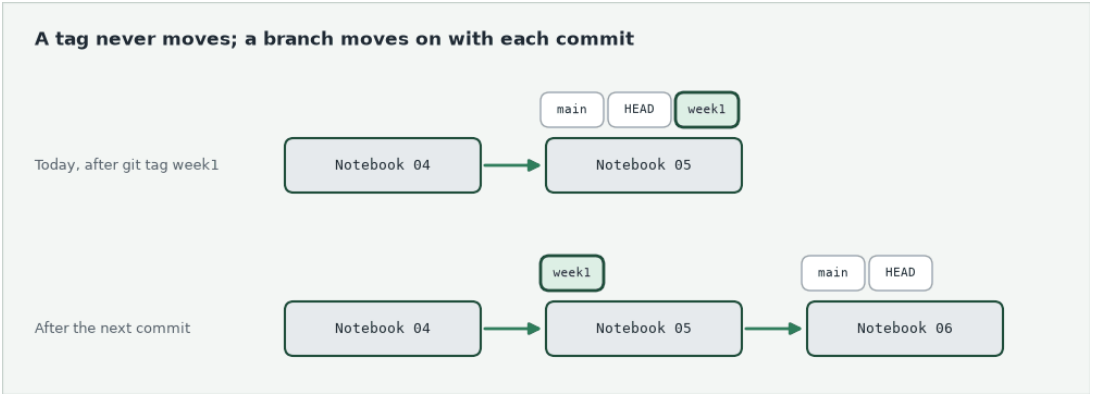
```
.............   [100%]
13 passed in 0.50s
```

```
!git -C "{work}" status --short
!git -C "{work}" diff --stat
 M test_bondmath.py
?? __pycache__/
 test_bondmath.py | 15 +++++++++++++++
 1 file changed, 15 insertions(+)
```

Only the test file changed, by 15 added lines; "bondmath.py" is untouched, and "__pycache__" waits for notebook 06.

```
subject = 'Notebook 05: tests of par pricing and of price against yield'
body = 'The new issue calendar relied on both rules; the tests pin them down before week 2.'
!git -C "{work}" add test_bondmath.py
!git -C "{work}" commit -q -m "{subject}" -m "{body}"
```

```
!git -C "{work}" tag week1
!git -C "{work}" log --oneline --decorate
0a77531 (HEAD -> main, tag: week1) Notebook 05: tests of par pricing and of price against yield
7147a11 Notebook 04: yield_from_price
3f57cac Notebook 03: convert_yield
408a678 Notebook 02: cash_flows and price_from_yield
d029474 Notebook 01: future_value and present_value
```



A new commit moves "main" on; "week1" stays where it is. In Excel terms, it is a copy saved as "calendar_week1.xlsx" while the live file moves on.

13 tests pass and "week1" marks the module as the week ends. Last ask: the workbook.

5.6 A workbook Excel calculates itself

Course 1's "to_excel" writes a table to a workbook, and a text that starts with "=" goes in as an Excel formula. So the yield and price columns can be live "YIELD" and "PRICE", beside Python's numbers.

```
sheet = calendar[['cusip', 'issuer', 'coupon_pct', 'maturity', 'reoffer_price',
                  'yield_pct']].copy()
sheet['maturity'] = pd.to_datetime(sheet['maturity'])
rows = range(2, 10)
sheet['excel_yield_pct'] = [f'=YIELD(DATE(2026,9,30),D{r},C{r}/100,E{r},100,2,0)*100'
                            for r in rows]
sheet['yield_gap'] = [f'=G{r}-F{r}' for r in rows]
sheet['price_at_5_50'] = calendar['price_at_5_50']
sheet['excel_price_at_5_50'] = [f'=PRICE(DATE(2026,9,30),D{r},C{r}/100,5.5/100,100,2,0)'
                                for r in rows]
sheet['price_gap'] = [f'=J{r}-I{r}' for r in rows]
print(sheet.loc[0, 'excel_yield_pct'])
=YIELD(DATE(2026,9,30),D2,C2/100,E2,100,2,0)*100
```

```
sheet.to_excel('new_issue_calendar.xlsx', index=False, engine='openpyxl')
print(f'Open it from PowerShell: Invoke-Item "$env:TEMP\\{work.name}\\new_issue_calendar.xlsx"')
```

Change a re-offer price in column E, and its yield and gap change with it.

Run the PowerShell line the save cell printed; in Colab, download the file and choose Enable Editing if Excel opens it in Protected View.

Job	Excel	Result
Yield, row 2	`=YIELD(DATE(2026,9,30),D2,C2/100,E2,100,2,0)*100`	5.153908
Price at 5.50, row 2	`=PRICE(DATE(2026,9,30),D2,C2/100,5.5/100,100,2,0)`	98.379986
Largest yield gap	`=G4-F4`	-6.1E-09
Largest price gap	`=K8, from =J8-I8`	1.3E-13

Excel calculated the file itself and agreed to 6.1e-09 percent. The note is answered.

The checks behind every answer, and bring it home

Summary

Check	Result
Calendar typed	8 issues, 8 CUSIPs, coupons on the grid
yield_from_price against YIELD, 8 rows	within 6.1e-09 percent
price_from_yield against PRICE at 5.50, 8 rows	within 1.1e-13
Eighth steps against PRICE, 8 rows	within 2.8e-14
pytest	13 passed
Workbook opened in Excel	gaps at most 6.1e-09 percent and 1.3e-13

Copy the work folder's two files into your own folder, then test, commit, and tag there. In a PowerShell terminal in VS Code, with the course's ".venv" active, use the work folder name your setup cell printed:

```
Set-Location "$HOME\projects\my_bondmath"
$work = "$env:TEMP\pfi_course2_05_k4w7q2xr"
Copy-Item "$work\bondmath.py", "$work\test_bondmath.py" .
python -m pytest -q
git add bondmath.py test_bondmath.py
git commit -m "Notebook 05: tests of par pricing and of price against yield"
git tag week1
```

The copy replaces your two files, so anything of your own goes into the work folder first. In Colab, download the two files from the file browser instead.

You Can Now

Turn a calendar of re-offer prices into yields, one bond at a time.

Find the coupon on the grid of eighths that prices nearest par.

Measure what an eighth of coupon is worth, and say why it is under an eighth a year.

Name a commit with "git tag", and read the names with "git log --oneline --decorate".

Predict first: a point below par, and eight in one call

Predict 1

Corvolvey (7 percent, 5 years) and Rulvolvey (6 percent, 30 years) are both re-offered about a point below 100. Corvolvey yields 24.2 bp over its coupon. How far over its coupon is Rulvolvey?

A) Near 4.0 bp: six times the term, a sixth of the gap.

B) Near 7.5 bp: the far years count for less.

C) Near 24.2 bp: the same discount, the same gap.

```
for ticker in ['CORY', 'RULY']:
    issue = calendar[calendar['ticker'] == ticker].iloc[0]
    issue_yield_pct = bondmath.yield_from_price(issue['coupon_pct'], issue['reoffer_price'],
                                                issue['years'])
    gap_bp = (issue_yield_pct - issue['coupon_pct']) * 100
    print(f"{ticker}, {issue['years']} years: yield {issue_yield_pct:.4f}, {gap_bp:.1f} bp over")
```

Predict 2

Hand the three whole columns to "yield_from_price" at once. What stops it?

A) An "if" in the module: it needs one True or False, and a column gives eight.

B) The "" power: a whole column cannot be raised to a power.

C) The type hints: a column is not a "float".

```
bondmath.yield_from_price(calendar['coupon_pct'], calendar['reoffer_price'], calendar['years'])
```

Predict first: two bonds at 5.50, and two grid coupons

Predict 3

Corvolvey pays 7 percent for 5 years, Rulvolvey 6 percent for 30. At 5.50 percent, whose price is higher?

A) Rulvolvey: its extra coupon lasts six times longer.

B) Corvolvey: a full point more coupon.

C) The same: one yield, one price.

```
common_yield_pct = 5.5
for ticker in ['CORY', 'RULY']:
    issue = calendar[calendar['ticker'] == ticker].iloc[0]
    price = bondmath.price_from_yield(issue['coupon_pct'], common_yield_pct, issue['years'])
    print(f"{ticker}: {issue['coupon_pct']} percent for {issue['years']} years, price {price:.4f}")
```

Predict 4

On Galvostrin's 10-year term, at its own yield, which coupon prices nearer 100?

A) 5.625, the calendar's coupon.

B) 5.750, the grid coupon nearer the yield.

C) Both equally near.

```
for coupon_pct in [low_coupon, high_coupon]:
    price = bondmath.price_from_yield(coupon_pct, galvostrin['yield_pct'], galvostrin['years'])
    print(f'Coupon {coupon_pct:.3f}: price {price:.4f}, {abs(price - 100):.4f} from 100')
```

Predict first: an eighth, and reading it back

Predict 5

Rulvolvey at 5.50 percent, with a coupon of 6.125 instead of 6.000. An eighth a year for 30 years would be 3.75, but each payment is discounted. How much does the price rise, per 100? Write down a number.

```
price_now = bondmath.price_from_yield(6.0, common_yield_pct, 30)
price_stepped = bondmath.price_from_yield(6.125, common_yield_pct, 30)
print(f'One eighth of coupon adds {price_stepped - price_now:.4f} per 100 of face')
```

Predict 6

"pd.read_excel" reads the saved workbook back. What does its "excel_yield_pct" column hold?

A) The yields.

B) The formula text, starting with "=".

C) Nothing: empty cells.

```
back = pd.read_excel('new_issue_calendar.xlsx')
back[['cusip', 'yield_pct', 'excel_yield_pct', 'price_at_5_50', 'excel_price_at_5_50']].head(3)
```

The note, answered

The note asked	Answer
Yield at re-offer	4.6459 percent (Kezerone) to 9.0797 (Morvithe); 2.1 to 33.0 bp over the coupon, Elvorin at par
One yield for all	At 5.50, five above 100, two below, Hulvimer at 100; Rulvolvey 107.3057 over Corvolvey 106.4801
Coupon nearest par	The grid coupon nearest the yield: their own on four, one to three eighths up on the rest
An eighth's worth	0.3414 to 1.8264 per 100 at 5.50
A workbook Excel calculates	new_issue_calendar.xlsx, live YIELD and PRICE, gaps at most 6.1e-09 percent

Be proud: week 1's module prices and yields any bullet bond on a coupon date, and Excel agrees with every number.